Session 12 Homework — Guard a Read, Not Just a Write

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

By the end of class, every real write route on both Projects and Workspaces is protected by `InternalToolGuard`, logged, and documented through Swagger. One route was deliberately left open: `GET /projects/stats`. This homework asks you to make a real, independent judgment call — not every real access-control decision is about writes — and apply the identical Guard pattern to a read, reusing only what class already built.

Goal

`GET /projects/stats` becomes internal-tool-only, proven both ways (reject without the key, succeed with it), documented in Swagger exactly like every other guarded route, and observable through a real, meaningful success log.

What to build

Apply `InternalToolGuard` to `GET /projects/stats` (CORE); extend the same `@ApiResponse` documentation already used on every other guarded route to cover this one AND every route that didn't get it live in class (STRETCH); and write three short reasoning answers directly on `InternalToolGuard` itself (CHALLENGE).

CORE (required)

Protect `GET /projects/stats` in `ProjectsController` / `ProjectsService`:

- `@UseGuards(InternalToolGuard)` and `@ApiHeader(INTERNAL_TOOL_HEADER)` on the `stats` handler — the identical pattern already applied live to `create` / `update` / `remove`.
- `@ApiOperation({ summary: 'Count how many projects exist (internal tool only)' })` — keep the summary honest about the new requirement.
- In `ProjectsService`'s own `stats()`, add one meaningful success log line (`this.logger.log(...)`) that states the real count returned — the same observability habit as every other mutating operation this session already logs.

STRETCH (required)

Using only syntax already introduced during this session, improve Swagger documentation on every protected endpoint:

- Add `@ApiResponse` for both the real success status and the real `403` on `GET /projects/stats`.

- Confirm every other already-guarded route (Projects' create/update/remove, all three of Workspaces' own guarded routes) documents both its real success status and its `403` — add whatever is still missing.
- Never expose the actual internal key value anywhere in Swagger — only that the header is required.

CHALLENGE (required)

Write three short, direct answers as a comment block on `InternalToolGuard` itself, directly above the class:

1. Why is the internal-tool header rule access control but NOT authentication?
2. Why is a Guard safer than repeating the same header check inside every controller method?
3. What would fundamentally change later, once real JWT authentication and real users are introduced?

Rules

Allowed: everything class already used — `@UseGuards`, `@ApiHeader`, `@ApiResponse`, `ConfigService`, Nest's own `Logger`. No new syntax this homework introduces anything beyond what live CORE and teamwork already built. **Not yet:** no JWT, no users, no roles, no custom decorators, no interceptors, no database-level uniqueness constraints or indexes, no deeper Mongoose query patterns.

Test it

BASH

```
curl -i localhost:3000/projects/stats
curl -i localhost:3000/projects/stats -H "x-internal-tool-key: <your real local .env value>"
```

Then open `http://localhost:3000/docs`, expand `GET /projects/stats`, and confirm it documents the required header and both its real response codes — the same way every other guarded route already does.

Done when

- `GET /projects/stats` with no header (or the wrong one) returns a clean `403`.
- `GET /projects/stats` with the real key returns `200` and the real `{"total": N}` count.
- The server log shows a real, meaningful success line only on the successful request — the rejected request produces no `ProjectsService` log line at all.
- Swagger's own `/docs` page documents `GET /projects/stats`'s required header and both of its real response codes, exactly like every other guarded route.
- Every already-guarded route on both resources documents both its real success status and its `403` — nothing guarded is left undocumented.
- All three CHALLENGE answers explain WHY, not just WHAT — matching the standard this course has held every prior CHALLENGE tier to.

Working time

About 90–120 minutes for a normal student — CORE is the same Guard pattern applied once more, STRETCH is documentation completeness across already-written code, and CHALLENGE is three short written answers, not new code.